

What is Prompt Caching? Optimize LLM Latency with AI Transformers

Understand how caching the input to an LLM — not its output — cuts cost and latency on long, repeated prompts.

For developers building LLM applications who want to cut API latency and cost on prompts with large, repeated context · 27 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

[Watch the source video](#)[Read the full lesson](#)[Read the condensed version](#)

What matters

- ✓ Prompt caching stores the model's internal representation of the input (KV pairs), not the output — every cached request still generates a fresh response.
- ✓ This is different from output caching, which stores the final answer and skips the model call entirely for identical repeated queries.
- ✓ Before generating any output, a transformer computes key-value pairs for every token at every layer; this is called the prefill phase, and it is the part prompt caching speeds up.
- ✓ Caching only pays off once a prompt is large — short prompts finish the prefill phase so fast that managing a cache costs more than it saves.
- ✓ Common things to cache are large reference documents, system prompts, few-shot examples, tool/function definitions, and conversation history — anything static that gets reused across requests.
- ✓ Automatic caching relies on prefix matching: it compares the new prompt to the cached one token by token from the start and stops at the first difference, so static content must come before dynamic content (like the user's question) or the cache never gets used.
- ✓ Most systems require a prompt to be around 1024 tokens or more before caching activates, and cached entries typically expire after 5 to 10 minutes (some last up to 24 hours) to keep data fresh.
- ✓ Some providers cache automatically based on prefix matching; others require you to explicitly mark which segments of a prompt should be cached in the API call.

What it covers

01 Prompt Caching Is Not Output Caching

0:00

Prompt caching stores what the model computed from your input, not the answer it produced — every request still gets a freshly generated response.

02 Inside the Transformer: KV Pairs and the Prefill Phase

2:07

Before an LLM can write its first output token, it converts every input token into key-value pairs at every transformer layer — that conversion is the expensive step prompt caching skips.

03 Why Caching Pays Off with Long, Reused Context

3:36

Caching a large, unchanging block of context — like a document pasted into the prompt — lets later requests skip straight to processing only the new question.

04 What Gets Cached in Practice

5:05

Anything static and reused — reference documents, system prompts, few-shot examples, tool definitions, and conversation history — are the usual candidates for caching.

05 **Prefix Matching: How Caching Decides What to Reuse** 5:47

Automatic caching compares a new prompt against the cached one token by token from the very start, and stops reusing the cache the instant a token differs — so prompt order determines whether caching helps at all.

06 **Practical Limits: Minimum Size, Expiration, and Explicit Caching** 7:58

Caching only pays off above a minimum prompt size, cached entries expire after minutes to hours, and some systems require you to mark cacheable sections explicitly rather than relying on automatic prefix matching.

Glossary

Prompt caching — Storing the model's internal processing of an input prompt (or its static prefix) so that repeated or shared portions of future prompts don't need to be recomputed. The model still generates a fresh output every time.

KV pairs (key-value pairs) — Internal values a transformer computes for every token at every layer, representing how each token relates to the others in the input. These are what prompt caching actually stores.

Transformer layer — One of several stacked processing stages in a transformer model; KV pairs are computed independently at each layer as the input passes through the model.

System prompt — Instructions sent with every request to an LLM that define its behavior, persona, and rules, typically identical across many calls and therefore a strong caching candidate.

Context window — The maximum amount of text (measured in tokens) an LLM can process as input in a single request.

Output caching — Storing the final generated response to a prompt and returning it directly for identical future requests, skipping the model call entirely.

Prefill phase — The stage of LLM processing where the model computes KV pairs for the entire input prompt, before it generates its first output token.

Prefix matching — A technique that compares a new prompt against a cached one token by token from the start, treating everything up to the first differing token as a cache hit.

Few-shot examples — Sample input/output pairs included in a prompt to show the model the format or style of response desired.

Token — A chunk of text (roughly a word or part of a word) that an LLM processes as a single unit; prompt length and caching thresholds are measured in tokens.

Explanations are AI-generated. Check anything you intend to rely on.